

Training quantum neural networks for T1-limited hardware: what the qg symmetry filter does and does not do

Vicente Humberto Monteverde

Universidad del Museo Social Argentino (UMSA), Argentina

ORCID 0000-0001-8884-4811

Draft of 3 October 2026

Abstract

A quantum neural network (QNN) built from gates that conserve the Hamming weight keeps its data in one weight sector, so the shots that left the sector can be discarded at readout. This is the qg filter of the qang library [1, 2]. We show that, with equal amplitude damping (T1) on every qubit, the filtered readout of such a network is exactly the noiseless one in any weight sector. The kept fraction of shots is $(1 - \gamma)^{kd}$ for weight k and depth d , and training under T1 with the filter gives exactly the parameters of noiseless training. We then measure what this is worth on four small classification datasets. Every result is reported with the filter and without it, and every prediction was committed before its run, failures included. Over 60 runs per model, a network trained on a simulator and run under T1 gains +2.5 points from the filter at weight 1 (95% CI [0.8, 4.1]) and +16.5 points at weight 2 ([11.8, 21.2]), recovering its noiseless accuracy exactly. A network trained under the calibrated noise without the filter catches up (differences of 0.1 points), and the filter corrects neither dephasing nor unequal T1 across qubits, although alone it loses less than one point up to a $\pm 80\%$ spread of $1/T_1$. With noise-aware training and a richer readout, the model without the filter is 1.2 points better: discarding decayed shots can cost accuracy. The weight-2 deficit comes from the pair-product encoding, not from the readout. On noise models of IBM and IonQ devices the filter cuts the error of the measured qg_Z by 2.9–3.6 $\times$. These models are classically simulable and no quantum advantage is claimed. The filter is a way to make noise-free training valid under relaxation, at a shot cost known in advance. The models are released as `qang.qml` (version 0.6.0, `pip install qang`).

1 Introduction

Variational quantum classifiers are trained on simulators and run on noisy devices, or trained directly under device noise. Noise-aware training [3] is the usual remedy when the two differ. Symmetry verification [4, 5] is a different remedy: when the ideal circuit conserves a quantity, outcomes that violate it can be discarded. The qang project expresses qubit readouts in qg units ($qg_Z = \langle \sigma_z \rangle$ and its companions) [1]. It built a register-mean witness and a shot-level filter for circuits that conserve a Hamming weight [2]. This note asks a narrow question: for a QNN that conserves the weight, how much does that filter change the accuracy of a classifier on T1-limited hardware, and when does it stop mattering?

We answer with exact simulation, a fixed protocol and an explicit comparison. Every reading is taken with qang (filtered) and without it (raw), on the same trained parameters or the same initialization. The predictions of every study were committed to the repository before the run that tested them. Of 44 pre-registered predictions in the studies summarized here, 33 passed and 11 failed; the failures are reported alongside the passes.

2 Model and theory

Weight-conserving QNN. On n qubits, a reconfigurable beam splitter (RBS) gate acts on a pair (a, b) as a Givens rotation between $|10\rangle$ and $|01\rangle$ and leaves $|00\rangle$ and $|11\rangle$ fixed, so it conserves the Hamming weight [6]. We use $n = 5$ and three layers of RBS gates on the pairs $\{(0, 1), (2, 3)\}$, $\{(1, 2), (3, 4)\}$, $\{(4, 0)\}$. That gives 15 angles and depth $d = 9$ (sublayers). The readout is $z = \sum_i c_i qg_Z^{(i)} + b$ with trainable c_i, b and a logistic loss.

Encodings. With four features $x \in [-1, 1]^4$ let $v = (x, 1)/\|(x, 1)\|$; the constant component keeps $\|x\|$. Model E (weight 1) puts v_i on the basis state with qubit i excited. Model W (weight 2) puts $v_i v_j$ on the state with qubits $i < j$ excited, normalized. The sectors have dimensions 5 and 10.

The filter. With qang, the readout keeps only the shots whose weight equals the input weight k and renormalizes (`qang.sectors.filter_distribution`). Without qang, all shots are used.

F1/F4: exactness under equal T1. Let amplitude damping with probability γ act on every qubit after every sublayer. Its no-jump Kraus operator multiplies a basis state with k excitations by $(1 - \gamma)^{k/2}$; the same factor for every state in the weight- k sector. Every jump lowers the weight and leaves the sector. The weight-conserving gates never bring a lowered state back. So, conditioned on the weight being k , the state is exactly the noiseless one, and

$$\text{kept fraction} = (1 - \gamma)^{kd}. \quad (1)$$

For $\gamma = 0.08$ and $d = 9$ this is 0.472 at weight 1 and 0.223 at weight 2. If the damping differs between qubits, the no-jump factor is $\prod_{q \in S} (1 - \gamma_q)^{1/2}$ over the excited set S , which is not uniform, and exactness is lost. Dephasing conserves the weight, so the filter cannot see it.

F3: training. Since the filtered T1 readout equals the noiseless one for every parameter value, the loss and all gradients are identical. Training under T1 with the filter therefore yields the parameters of noiseless training from the same initialization.

F2: simulability. At weight 1 the network is a 5×5 orthogonal matrix O on v , and z is a quadratic form in v . At weight k the sector has dimension $\binom{n}{k}$, polynomial for fixed k . These models can be simulated classically. Whether larger, higher-weight versions could not be is a separate question [7] that this note does not address.

3 Methods

Datasets (binary): iris (versicolor vs virginica), breast cancer, wine (classes 0 vs 1) and digits (3 vs 8), with 100 samples per class where available. Each run uses a stratified 70/30 split, standardization, PCA to four components where needed, and scaling to $[-1, 1]$; one test sample is worth 2–3 points. Training uses Adam for 120 epochs, with central-difference gradients on the angles. Noise conditions are applied after every sublayer: equal T1 with $\gamma = 0.08$; unequal T1 with γ_q from 0.04 to 0.12 (mean 0.08); and equal T1 plus a phase flip with probability 0.03. The simulation is exact. T1 and dephasing keep the density matrix block-diagonal in the Hamming weight, so one block per weight is propagated; this is checked against the full density matrix to 10^{-12} . Protocol: each study’s script, with its predictions written in the header, was committed before the seed of its reported run was used; code was debugged on another seed with few epochs. The commit hashes are in the research notes [1].

4 Results

Single-seed studies. The weight-conserving QNN that keeps the norm reaches a mean accuracy of 0.960 over the four datasets. That is level with a standard (non-conserving) four-qubit QNN (0.952) and with an RBF support-vector machine (0.957) and a small MLP (0.961). Without the norm component the same network gets 0.791. Trained noiselessly and run under T1 with the filter, it keeps exactly its noiseless accuracy. Training under T1 with the filter reproduces noiseless training to 5×10^{-9} in the parameters (F3). At $\gamma = 0.08$, a standard QNN trained noiselessly collapses under T1 (0.771); trained under T1 it recovers (0.944).

The predictions that failed:

- arccos encoding gave no accuracy gain;
- the classical quadratic twin did not match the QNN within 1 point at default regularization;
- unequal T1 cost the filter 1.5 points in one run (bound: 1 point);
- weight 2 was less accurate than weight 1 (by 3.7 points);
- trained under T1, the weight-2 model without the filter was 0.6 points ahead of the filtered one;
- in the studies of Sec. 4: the $\langle Z_i Z_j \rangle$ readout did not help weight 2 (two predictions), the filter was not neutral under noise-aware training with that readout, the gain of the filter at one T1 spread had a CI touching zero, weight 2 was not more sensitive to the spread than weight 1, and the pair-product encoding had a lower information ceiling than the unary one.

Replication over 60 runs. The key comparisons were repeated over $5 \text{ seeds} \times 3 \text{ splits} \times 4 \text{ datasets}$. Table 1 and Fig. 1 give the results; all five pre-registered predictions of this replication passed.

reading	E (weight 1)		W (weight 2)	
	with qang	without	with qang	without
exact (no noise)	0.952		0.922	
T1, trained without noise	0.952	0.928	0.922	0.757
T1, trained under the noise	0.952	0.951	0.922	0.924
unequal T1, trained without noise	0.947	0.921	0.916	0.732
dephasing, trained without noise	0.918	0.792	0.882	0.656
dephasing, trained under the noise	0.950		0.926	

Table 1: Mean test accuracy over 60 runs per model, with and without the qg filter.

The paired differences (mean, 95% CI across seeds) were as follows.

- *Gain from the filter, trained without noise, under T1:* +2.5 [0.8, 4.1] points (E) and +16.5 [11.8, 21.2] (W). The filtered model was ahead or tied in 50/60 (E) and 56/60 (W) runs.
- *With noise-aware training:* +0.1 [−0.6, 0.9] (E) and −0.1 [−0.8, 0.5] (W).
- *Loss from unequal T1:* 0.5 [−0.2, 1.2] (E) and 0.7 [0.0, 1.3] (W). The 1.5 points of the earlier run were at the top of the spread.
- *Dephasing, noise-aware minus clean training with the filter:* +3.2 (E) and +4.5 (W), both CIs above 0.

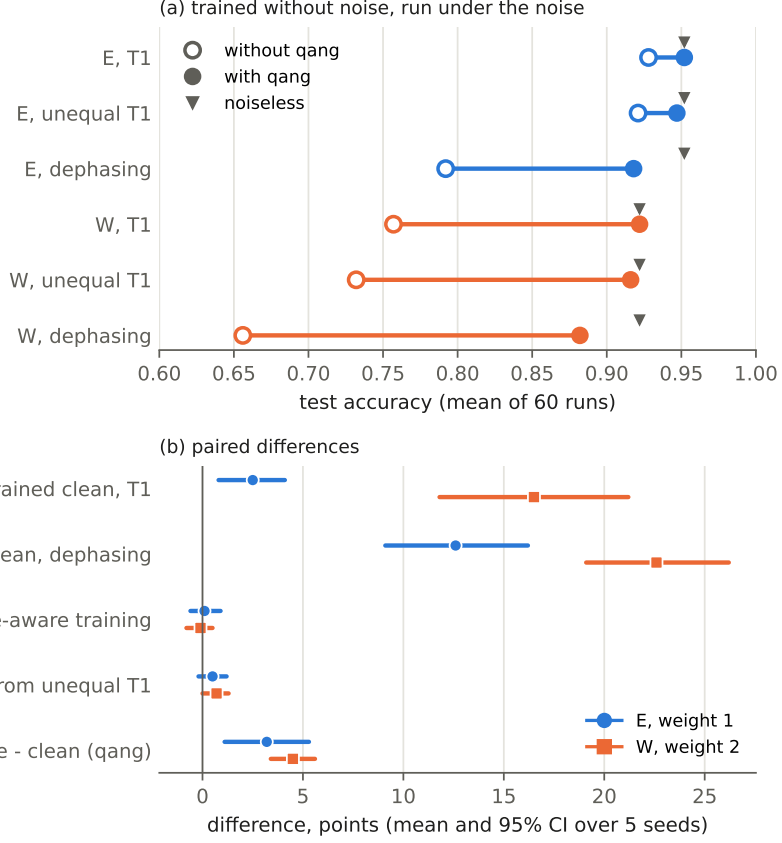


Figure 1: (a) Accuracy of networks trained without noise and run under the noise, without (open) and with (filled) the filter; triangles mark the noiseless accuracy. (b) Paired differences, mean and 95% CI across 5 seeds.

- *Under dephasing, a model trained without noise still gains +12.6 (E) and +22.6 (W) points from the filter, which removes the T1 part of the combined noise.*
- *Weight 2 against weight 1: -3.0 $[-4.0, -2.0]$ points.*

Readout, encoding and T1 spread (60 runs each).

- *Correlation readout.* Adding the $\binom{5}{2} = 10$ correlations $\langle Z_i Z_j \rangle$ makes the weight-2 readout a linear function of the whole sector distribution (rank 10 instead of 5). It did not help: -0.6 $[-1.5, 0.4]$ points. Trained under T1 with 15 readout weights, the model *without* the filter was 1.2 $[0.4, 2.0]$ points better. The decayed shots still carry information about where the excitations were, and the filter discards it. The filter is still exact for this readout, and trained without noise it adds +14.9 points.
- *Encoding.* The best unconstrained quadratic form in the pair-product amplitudes reaches 0.928, 2.4 $[1.7, 3.0]$ points below the same ceiling for the unary amplitudes (0.952). The normalization of the products distorts the data. Loading the weight-1 data on five weight-2 states instead (a “ring” of pairs $(k, k+1)$) gives 0.943, within 1.0 point of weight 1 and 2.3 $[0.7, 3.9]$ points above the pairs. With this encoding the gain of the filter under T1 is the largest seen, +23.9 points.
- *T1 spread.* With $\gamma_q = 0.08(1 + s u_q)$ and u_q spread over $[-1, 1]$, the loss of the filter was 0.3/0.2 points (weight 1/2) at $s = 0.5$ and 1.4/1.3 at $s = 1$. The filter alone keeps the loss

below one point up to about $s \approx 0.8$, i.e. a $\pm 80\%$ spread of the decay rates $1/T_1$ around their mean.

Device noise models. The weight-1 model was compiled to Qiskit (a cascade of 4 RBS gates loads the signed unary amplitudes; 15 trained RBS gates follow; the circuit matches the model to 10^{-16}). It was run on 30 iris test inputs with 1000 shots each, read with and without the filter from the same shots. The noise models were those of three IBM devices (calibration-based fake backends simulated by Aer) and the IonQ cloud simulator with the Aria-1 and Forte-1 noise models. On all five the accuracy was 0.933 with and without the filter, equal to the noiseless accuracy: the margins of this model are wide. The error of the measured qg_Z fell from 0.138–0.206 without the filter to 0.040–0.072 with it ($2.9\text{--}3.6\times$), keeping 69–74% of the shots. These are noise models, not devices; the hardware runs are prepared, with cost confirmations, and pending.

5 Discussion

The difference the filter makes depends on how the network was trained. Trained on a simulator and run under relaxation, the filter is decisive. Its effect grows with the weight, because a weight- k state decays k times faster. Trained under a calibrated noise model, a network without the filter learns the decay, and the filter adds nothing measurable. What the filter provides is therefore procedural rather than an accuracy gain over the best alternative. Under equal T_1 it makes noise-free training exactly valid, with no noise model and no noisy training runs, and its cost (the discarded shots) is known before running. Unequal T_1 and dephasing are not corrected. On hardware with a large spread of T_1 or strong dephasing, the filter complements a noise model and does not replace it. Two practical rules follow from the studies above. Up to about a $\pm 80\%$ spread of $1/T_1$ across qubits, a network trained on a simulator can be run with the filter alone, losing less than one point. When the network is instead trained under the calibrated noise, and especially when its readout has many parameters, the decayed shots are useful data, and the filter can cost about a point. For weight-2 models, how the data are loaded matters more than what is read out.

Limitations. All noise is simulated, including the device noise models. There is one architecture per weight, five qubits, and small datasets in which one test sample is worth 2–3 points. The ring encoding uses only five of the ten weight-2 states, so a better use of the larger sector is still open. No quantum advantage is claimed: these networks are classically simulable. A hardware run with the filter is the natural next test.

Code and data availability

Models: `qang.qml` in `qang 0.6.0` (`pip install qang`), NumPy only, with `WeightQNN` (weights 1 and 2, readouts "z" and "zz", encodings "pairs" and "ring"), `kept_fraction` and `compare_qang`. Hardware-ready script: `examples/qnn_hardware_qg.py` (IBM and IonQ). Studies: `examples/qnn*_qg.py` and RESEARCH_NOTES §75–§82, §85 and §87, with tests in `tests/`. The Colab notebooks `notebooks/qang.qml.ipynb` (English) and `qang.qml_es.ipynb` (Spanish) install the library from PyPI. Repository: <https://github.com/Viny2030/qang>.

```
from qang.qml import WeightQNN
m = WeightQNN(n_qubits=5, weight=2).fit(X_train, y_train)
m.score(X_test, y_test, gamma=0.08, qang=True) # = noiseless accuracy
m.score(X_test, y_test, gamma=0.08, qang=False) # raw readout
```

References

- [1] Vicente Humberto Monteverde. The Qang (qg): A unified angular-probability unit and metric for parametric quantum circuit design. 2026. doi: 10.5281/zenodo.22832150.
- [2] Vicente Humberto Monteverde. The register-mean polar bias as a symmetry witness: error mitigation, constrained optimization and qubit characterization in qg units. *Zenodo preprint*, 2026. doi: 10.5281/zenodo.23036963.
- [3] Hanrui Wang, Jiaqi Gu, Yongshan Ding, Zirui Li, Frederic T. Chong, David Z. Pan, and Song Han. QuantumNAT: quantum noise-aware training with noise injection, quantization and normalization. In *Proceedings of the 59th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6, 2022. doi: 10.1145/3489517.3530400.
- [4] Xavier Bonet-Monroig, Ramiro Sagastizabal, M. Singh, and Thomas E. O’Brien. Low-cost error mitigation by symmetry verification. *Physical Review A*, 98:062339, 2018.
- [5] Sam McArdle, Xiao Yuan, and Simon Benjamin. Error-mitigated digital quantum simulation. *Physical Review Letters*, 122:180501, 2019.
- [6] Jonas Landman, Natansh Mathur, Yun Yvonna Li, Martin Strahm, Skander Kazdaghli, Anupam Prakash, and Iordanis Kerenidis. Quantum methods for neural networks and application to medical image classification. *Quantum*, 6:881, 2022. doi: 10.22331/q-2022-12-22-881.
- [7] M. Cerezo et al. Does provable absence of barren plateaus imply classical simulability? Or, why we need to rethink variational quantum computing, 2023. arXiv:2312.09121.